

PATERNI PONASANJA

State Pattern [implementacija]

State patern omogućava objektu da mijenja svoje ponašanje u zavisnosti od stanja u kojem se trenutno nalazi. Ovaj patern je pogodan za sisteme u kojima objekti prolaze kroz više različitih stanja, pri čemu svako stanje definiše različita pravila ponašanja i dozvoljene operacije.

U sistemu za prodaju i iznajmljivanje računarske opreme ovaj patern može biti primijenjen na upravljanje rezervacijama i transakcijama. Rezervacija može imati statuse *Na čekanju*, *Potvrđena*, *Otkazana* i *Završena*, dok transakcija može imati statuse *Na čekanju*, *Završena* i *Otkazana*. U zavisnosti od trenutnog statusa dozvoljene su različite akcije. Na primjer, rezervacija koja je već potvrđena ne može ponovo biti potvrđena, dok se otkazana rezervacija ne može završiti.

Implementacijom State paterna svako stanje bi bilo predstavljeno zasebnom klasom koja definiše dozvoljene operacije i prelaze u druga stanja. Na taj način se izbjegava veliki broj uslovnih naredbi, a sistem postaje jednostavniji za održavanje i proširivanje.

Observer Pattern [implementacija]

Observer patern uspostavlja vezu između objekata tako da promjena stanja jednog objekta automatski obavještava sve objekte koji su zainteresovani za tu promjenu. Ovaj patern je posebno pogodan za sisteme koji koriste različite vrste obavještenja.

U našem sistemu osnovni način komunikacije sa korisnicima predstavlja elektronska pošta. Observer patern može biti iskorišten za slanje automatskih obavještenja prilikom promjene statusa rezervacije ili transakcije. Na primjer, kada prodavac potvrdi rezervaciju artikla, kupac automatski dobija obavještenje putem e-maila. Slično tome, korisnik može biti obaviješten o otkazivanju rezervacije, završetku transakcije ili promjeni statusa zahtjeva za uklanjanje artikla.

Implementacijom Observer paterna postiže se odvajanje logike poslovnih procesa od logike slanja obavještenja, čime sistem postaje pregledniji i lakši za održavanje.

Strategy Pattern

Strategy pattern koristi se kada za isti problem postoji više različitih algoritama ili načina izvršavanja određene funkcionalnosti, pri čemu korisnik ili sistem može odabrati odgovarajući algoritam u toku rada. Na taj način se pojedinačni algoritmi izdvajaju u zasebne klase, dok ostatak sistema ostaje nezavisan od njihove implementacije.

U trenutnoj verziji sistema za prodaju i iznajmljivanje računarske opreme ovaj pattern nije neophodan, ali bi mogao biti vrlo koristan u budućim nadogradnjama. Jedna od mogućih primjena bila bi implementacija različitih načina pretrage artikala. Korisnicima bi se mogla omogućiti pretraga po nazivu, kategoriji, cijeni, proizvođaču ili prosječnoj ocjeni proizvoda, pri čemu bi svaki način pretrage predstavljao zasebnu strategiju.

Također, Strategy pattern bi se mogao primijeniti prilikom obračuna cijene iznajmljivanja. Sistem bi u budućnosti mogao podržavati dnevni, sedmični i mjesečni najam, kao i različite modele obračuna za fizička i pravna lica. U tom slučaju bi svaki način obračuna bio predstavljen posebnom strategijom, dok bi ostatak sistema koristio jedinstven interfejs bez potrebe za izmjenama postojećeg koda.

Template Method Pattern

Template Method pattern koristi se kada više procesa ima istu osnovnu strukturu, ali se pojedini koraci razlikuju u zavisnosti od konkretne situacije. Osnovni algoritam definira se u nadklasi, dok se specifični koraci implementiraju u izvedenim klasama.

U budućem razvoju sistema ovaj pattern mogao bi biti iskorišten za obradu različitih tipova transakcija. Proces kupovine i proces iznajmljivanja imaju veliki broj zajedničkih koraka, kao što su provjera korisnika, provjera dostupnosti artikla, kreiranje transakcije i slanje obavještenja korisnicima. Međutim, određeni dijelovi procesa razlikuju se u zavisnosti od vrste transakcije.

Na primjer, kod iznajmljivanja potrebno je evidentirati period trajanja najma i rezervisati artikal za određeni vremenski period, dok se kod kupovine nakon završetka transakcije artikal trajno uklanja iz ponude. Primjenom Template Method patterna zajednička struktura procesa ostala bi ista, dok bi se specifični koraci implementirali u posebnim klasama za kupovinu i iznajmljivanje.

Iterator Pattern

Iterator pattern omogućava prolazak kroz kolekciju objekata bez potrebe da korisnik poznaje način na koji su podaci organizovani i pohranjeni. Na taj način se postiže jednostavniji pristup podacima i veća fleksibilnost sistema.

U budućim verzijama sistema ovaj pattern mogao bi se koristiti za pregled velikog broja artikala, transakcija, rezervacija i korisnika. Kako broj podataka raste, javlja se potreba za efikasnim prolaskom kroz različite kolekcije i generisanjem izvještaja.

Jedna od mogućih primjena bila bi prikaz najbolje ocijenjenih artikala. Prilikom dodavanja novih ocjena sistem bi mogao koristiti Iterator pattern za prolazak kroz sve aktivne artikale i izračunavanje novih rang-lista. Slično tome, administrator bi mogao koristiti iterator za generisanje statističkih izvještaja o prodaji, iznajmljivanju ili aktivnosti korisnika bez poznavanja interne strukture podataka.

Command Pattern

Command pattern pretvara zahtjev za izvršavanje određene akcije u zaseban objekt koji sadrži sve informacije potrebne za izvršenje te akcije. Na taj način omogućava se evidentiranje izvršenih operacija, njihovo odgađanje ili eventualno poništavanje.

U trenutnoj verziji sistema ovaj pattern nije neophodan, ali bi mogao biti koristan u budućem razvoju administratorskog i moderatorskog dijela aplikacije. Akcije kao što su deaktivacija korisnika, odobravanje zahtjeva za uklanjanje artikla ili brisanje neprimjerenog sadržaja mogle bi biti predstavljene zasebnim komandama.

Na taj način sistem bi mogao voditi evidenciju svih izvršenih akcija, što bi bilo korisno za reviziju rada administratora i moderatora. Također, u budućnosti bi bilo moguće implementirati funkcionalnosti poput poništavanja određenih akcija ili izvršavanja više akcija u unaprijed definisanom redoslijedu.

Memento dizajn pattern

Memento dizajn patern omogućava čuvanje prethodnog stanja objekta bez narušavanja njegove enkapsulacije, te vraćanje objekta u ranije stanje kada je to potrebno. Ovaj patern je posebno koristan u sistemima gdje korisnici često vrše izmjene podataka i postoji potreba za poništavanjem ili vraćanjem prethodnih verzija.

U trenutnoj verziji sistema za prodaju i iznajmljivanje računarske opreme primjena ovog paternna nije neophodna, ali bi mogao biti veoma koristan u budućim nadogradnjama. Na primjer, prodavač može uređivati informacije o artiklu kao što su naziv, opis, cijena, slike ili dostupnost proizvoda. Prije svake izmjene sistem bi mogao sačuvati prethodno stanje artikla u obliku Memento objekta. U slučaju greške ili neželjene izmjene, prodavač ili administrator mogli bi vratiti artikal na prethodnu verziju bez potrebe za ručnim unosom podataka. Slična primjena moguća je i kod korisničkih profila, gdje bi se mogla čuvati istorija izmjena podataka korisnika, kao i kod administrativnih akcija koje utiču na sadržaj sistema. Na taj način povećala bi se sigurnost podataka i olakšalo upravljanje izmjenama unutar sistema.

Visitor dizajn pattern

Visitor dizajn patern omogućava dodavanje novih operacija nad postojećim objektima bez potrebe za izmjenom njihovih klasa. Ovaj patern je naročito pogodan kada postoji stabilna struktura objekata nad kojima se vremenom pojavljuje potreba za izvršavanjem novih funkcionalnosti.

U budućem razvoju sistema Visitor patern mogao bi biti iskorišten za generisanje različitih statističkih izvještaja i analiza poslovanja. Sistem sadrži više vrsta objekata kao što su korisnici, artikli, rezervacije i transakcije, a nad njima je moguće izvršavati različite analize.

Na primjer, jedan Visitor mogao bi izračunavati ukupan broj realizovanih prodaja, drugi broj aktivnih iznajmljivanja, treći prosječnu ocjenu artikala, dok bi četvrti generisao izvještaj o najaktivnijim prodavačima. Umjesto dodavanja posebnih metoda za svaku novu analizu u postojeće klase, svaka nova funkcionalnost bila bi implementirana kao zaseban Visitor.

Ovakav pristup omogućava jednostavnije proširivanje sistema u budućnosti, posebno ukoliko se ukaže potreba za naprednim izvještavanjem, analizom poslovanja ili generisanjem statistika za administratore sistema. Na taj način nove funkcionalnosti mogu biti dodane bez izmjena postojećih klasa, čime se poštuje Open/Closed princip objektno orijentisanog dizajna.